SCALER School of Technology · Priyansh Saxena

Building Agents: The Four Engineering Layers

Prompt ▪ Context ▪ Loop ▪ Harness ▪ A PR-review agent, improved four times

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 4 — Bonus Session /

ROADMAP

Agenda

1. The Agent as a Layered Stack
2. Prompt Engineering
3. Context Engineering
4. Loop Engineering
5. Harness Engineering
6. Putting It Together

TOC

Agenda

1. **The Agent as a Layered Stack**
2. **Prompt Engineering**
3. **Context Engineering**
4. **Loop Engineering**
5. **Harness Engineering**
6. **Putting It Together** — scoreboard + decision guide

Hour 2: The Practical

Live demo notebook: PR-review agent rebuilt three times. Scoreboard: F1, cost, steps, latency.

Two Quick Quizzes

After sections 2 and 5.

1

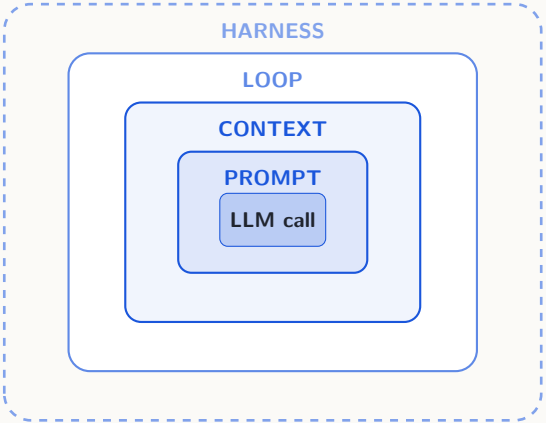
The Agent as a Layered Stack

1 Recap: Agent = LLM + Tools + Memory + Loop

- **LLM**: the reasoning engine; reads its input and decides what to do next
- **Tools**: structured functions; the LLM emits JSON calls; your code executes them
- **Memory**: what the agent remembers. In-context (cheap), external (slower), episodic (structured), semantic (facts)
- **Loop**: call $\rightarrow$ parse $\rightarrow$ act $\rightarrow$ observe $\rightarrow$ repeat. Adds flexibility but also risk (cost, loops, latency)

Today: why agents fail in production — and the four layers that fix it.

1 THE KEY DIAGRAM: Four Layers Around the LLM Call



Prompt	*what do we say?*
Context	*what's in the window?*
Loop	*how does output drive the next call?*
Harness	*what system does the loop run inside?*

Prompt = the sentence. Context = the page. Loop = the paragraph structure. Harness = the print shop.

They compose; they don't substitute.

2

Prompt Engineering

2 Anatomy of an Agent Prompt

1. **Role**: "You are a code review expert. You find real bugs, not style issues."
2. **Task**: "Review this PR (diff + files). Output a JSON list of findings."
3. **Constraints**: "Only report bugs in the changed lines or closely related code. Ignore unrelated issues."
4. **Tool-use policy**: "You have access to `read_file`, `search_codebase`. Always cite the exact line number."
5. **Output contract**: "Return JSON: {`findings`: [file, line, severity, category, explanation]}. No prose."
6. **Stop criteria**: "Stop when you've found all bugs or made 5 tool calls, whichever comes first."

*Observation: the prompt is a **recipe** — every ingredient matters. Missing even one causes failure.*

2 Techniques That Move the Needle

What works

- **Explicit JSON schema** + one-shot example
- **Reasoning elicitation**: "think before you call a tool"
- **Negative instructions**: "don't invent line numbers"
- **Few-shot**: 2-3 good examples, then the test case
- **Decomposition**: break the task into steps

When prompt hits its ceiling

The model only knows what's in front of it. A bug caused by code **outside the diff** is invisible no matter the wording.

Answer: go to context.

Reach for the prompt before fine-tuning. Prompt beats model weight on sample efficiency.

2 The Ceiling: Prompt Alone Cannot See Outside the Input

Example: TOCTOU bug (Time-of-check-time-of-use)

PR changes a cache check:

- `if cache.get(key): return cache[key]`

The diff is clean. No obvious bug.
But the **caller** (not in the diff) may have called cache.clear() in the meantime.

The prompt can't see it

No amount of "find TOCTOU bugs" will help if the enclosing function and the callers are not in the input. The model sees only the diff.
Wording alone can't fix a missing input — this bug needs the caller in the window, not a better sentence.

Motivates Layer 2: **Context Engineering**

3

Context Engineering

3 The Window Is a Budget, Not a Bucket

Tokens compete for attention

- A 100k-token context window is fixed — you have a budget
- Every token you add to historical context is a token you can't use for the current request
- Dumping everything is a loss strategy

Lost in the middle

- Models attend most to the **start** (system prompt) and **end** (current turn)
- Information in the **middle** is neglected
- Solution: put relevant content last

Production implication

A 10-turn agent at 4k tokens/turn $\approx$ 40k+ input tokens per run. At 1000 runs/day: 40M tokens/day from one agent.
Context engineering is how you cut the 4k without losing the bug.

3 Prompt vs. Context: The Reviewer Example

LEFT: Change the words (prompt layer)

- Change the role (“code review expert”)
- Write a better JSON schema
- Elicit reasoning (“think step-by-step”)
- Add few-shot examples

Same information as input.

Yields: *better formatting, fewer hallucinations, same information processed*

RIGHT: Change the information (context layer)

- Add the enclosing function
- Add the callers (so bugs like TOCTOU are visible)
- Put the diff last (most attended)
- Curate tool results (only the relevant snippet)

Same words in the prompt.

Yields: *new bugs found, higher recall, same cost/latency*

3 Context Engineering Techniques

- **Retrieve, don't dump**: pull only the diffs that matter; search by relevance, not size
- **Compact & rolling summary**: summarise old turns to fit more recent context; use compression
- **Order**: put the diff last; lead with context (enclosing functions, callers)
- **Curate tool results**: don't return 100 search hits; return the 3 most relevant
- **Scratchpad note-taking**: let the agent write to a working memory (visible on every turn) instead of appending to history
- **Memory types**: episodic (past actions), semantic (facts), working (current turn)
- **Prompt caching**: stable prefix (context that never changes) billed cheaper per re-read

Hint: prompt caching is a cost lever. If 50% of your input is stable, cache it and save 90% of the cost of those tokens on repeat calls.

3 The Economics: Why Context Matters at Scale

Back-of-envelope math

- One PR-review agent: 10 turns average
- 4000 tokens/turn = 40k+ input tokens per PR
- 1000 PRs reviewed per day
- = 40M tokens/day just from one production agent
- At $0.50/1M input tokens: $20/day for one agent

Context engineering cuts the 4k to 2k

- Caching saves 90% on repeated context
- Rolling summaries reduce history size
- Smart retrieval: pull only what matters
- Cost drops to $10/day; latency improves

The ceiling

Context still doesn't solve everything. Even with perfect context, the loop might need to revisit a decision or look at a file from a different angle.
Answer: loop engineering.

3 The Ceiling: One Call Can't Do Multi-Step Tasks

Example: Iterative code review

PR finds bug X. Agent suggests a fix. But implementing the fix breaks the callers of X. Now the agent needs to:

1. Find X (tool 1)
2. Understand the impact (search + read)
3. Revise the fix (think step-by-step)
4. Check the new fix (tool 2)

One LLM call can't do this

A single prompt-context pair forces a choice: be vague (and miss the impact) or be huge (and hit the cost budget fast).

Answer: give the agent a loop. Call → parse → act → observe → repeat.

Motivates Layer 3: **Loop Engineering**

4

Loop Engineering

4 The Loop: call → parse → act → observe → repeat

Pipelines run once

- Step 1 runs, then step 2, then step 3
- The path is fixed at design time
- If you need to go back, too bad

Agents loop

- Call the LLM → it picks a tool → you execute → you show the result
- LLM sees the result and picks the next tool
- Repeat until the LLM emits a final answer

Agents add a failure mode pipelines don't have

The loop might not stop.

- Infinite loops (calling the same tool forever)
- Oscillation (tool A → tool B → tool A)
- No progress (same state two turns in a row)

This is new risk. Price of flexibility.

4 Control Surfaces: How to Govern the Loop

Stopping the loop

- **Explicit stop signal**: LLM emits a final answer → stop
- **Hard step cap**: e.g. max 20 steps before forced termination
- **Loop detection**: track last N actions; if the same action repeats k times, abort
- **No-progress detection**: if nothing new learned in the last 2 steps, stop

What state carries forward

- Conversation history (full turn-by-turn)
- Scratchpad (working memory for current task)
- Tool results (append to context)

Re-plan and reflect

If the loop stalls, ask the LLM to reflect: "You've called these tools. What did you learn? Do you need a different approach?"
This is cheap (one more LLM call) and often unsticks the agent.

4 Loop Patterns: ReAct, Plan-Then-Execute, Evaluator-Optimizer

Pattern	When to use	Strength	Weakness
ReAct (Reasoning + Acting)	Exploratory tasks; code review	Interleaved thought + observation; grounded	Can wander; more LLM calls; latency unknown upfront
Plan-then-Execute	Multi-step workflows; known dependencies	One planning call, then linear execution	Brittle if the plan needs mid-course revision
Evaluator-Optimizer	Generation + refinement loops	Generate draft → evaluate → refine	Cost scales with iteration count

Practical advice: ReAct for exploration. Plan-and-execute for well-scoped workflows. Evaluator-optimizer when quality > speed.

4 The Ceiling: Loop Trusts the Model and Tools Completely

What the loop doesn't do

- Verify tool results (assumes output is correct)
- Validate model claims (assumes reasoning is sound)
- Check safety (assumes actions are appropriate)
- Audit permission (assumes tool access is correct)

The loop says: "How many times should we call tools?"
It doesn't say: "Should we believe this answer?"

Production failure

Agent confidently reports a bug on a comment line. The tool returned that result. The loop trusted it. No one checked.
Answer: harness engineering. Verification gates. Audit trails.

Motivates Layer 4: **Harness Engineering**

5

Harness Engineering

5 What Is the Harness?

The loop is the engine

- LLM reasoning
- Tool calling
- Iteration

The harness is the car

- Brakes (stop conditions, rate limits)
- Seatbelts (verification gates, sandboxing)
- Dashboard (observability, logging)
- Speed limiter (cost ceiling, timeout)

The harness is not the LLM

Everything the LLM **doesn't control**. The infrastructure, safety checks, and audit trail.
A demo agent is: loop + good prompt.
A production agent is: loop + prompt + harness.

5 The Seven Pieces of a Production Harness

1. **Tool design & sandboxing**: least privilege. Args schema-checked. No raw user input.
2. **Verification gates**: don't trust a claim. Check it. Agent says "bug on line 42" $\rightarrow$ verify it exists.
3. **Cost & rate ceilings**: hard budget per session. Alert on spikes.
4. **Human-in-the-loop**: risk-tier every action. High-risk needs approval.
5. **Observability**: full replayable trace. Every LLM call, tool call, state change.
6. **Error recovery**: retry logic, fallback responses, graceful degradation.
7. **Sub-agents**: isolate a noisy subtask. Let it fail independently.

All seven are non-negotiable in production. Pick your battles on prompt and context; don't skimp on the harness.

5 Demo Agent vs. Production Agent

Demo agent

- A loop with a good prompt
- Runs in a notebook or CLI
- You watch it in real-time
- Failure is: "Oops, that didn't work"
- Cost: who cares, it's a one-off

Production agent

- Loop + prompt + full harness
- Cost ceiling, rate limiting, audit trail
- Verification gates before high-risk actions
- Failure is: a customer gets wrong info or money moves incorrectly
- Cost: predicted, monitored, alerting on anomalies

The gap: the gap between demo and production is not "make it faster." It's "make it safe and predictable."

6

Putting It Together

6 QUIZ 1: Which Layer?

Q1: Findings are half prose, half JSON

Agent reports some findings as structured JSON and others as free text. Very inconsistent.
Answer: Prompt layer. Needs a clearer output contract and more examples.

Q2: Good answers but they cost 3¢ each and take 20 seconds

Every request works but the latency and cost are both too high. The agent needs to run faster.
Answer: Context layer. Reduce the input tokens (retrieve less, compress more). Enable prompt caching on the stable prefix.

Q3: Takes 30 steps on a simple task

One PR review takes 30 tool calls. Same simple task. Way too many steps.
Answer: Loop layer. Add loop detection, hard step cap (e.g. 10 steps), and re-plan on stall.

Q4: Confidently reported a bug on a comment line

Agent found a bug that doesn't actually exist — it's in a comment. No verification happened.
Answer: Harness layer. Add a verification gate: check the file and line exist and the code is actually there before reporting.

6 The Practical: One PR-Review Agent, Rebuilt Three Times

The fixture

- Toy Python project: 5 modules (cache, task queue, parser, retry, utils)
- 12 labelled PRs total; today we score **4** of them, each chosen to force one stage to earn its keep
- pr_01: off-by-one visible in the diff. pr_08: TOCTOU whose caller is **not** in the diff. pr_11: subtle enough that the agent double-checks it with `run_tests`. pr_10: clean — catches false positives.
- Scoring: precision + recall $\Rightarrow$ F1

The notebook

- Model: `gemini-3.1-flash-lite` (free tier) — run live, then cached
- Responses cached (offline replay in class)
- Same 4 PRs scored on each stage
- Metrics per stage: F1, $/run, steps, latency

Hour 2: live notebook walkthrough. You'll see each stage fail, then improve.

6 QUIZ 2: Which Layer? (Harder)

Q1: Every 5th run costs 5× more for no clear reason

Cost is unpredictable. Spiking every few runs. No pattern in the input.
Answer: Harness layer. Add cost tracing and a hard budget — probably a loop is triggering 1 in 5 times; add loop detection too.

Q2: Fails whenever the fix spans a 3-file call chain

Agent sometimes misses bugs that span multiple files. Only finds bugs in single-file changes.
Answer: Loop layer. It's not revisiting files after seeing tool results. Add a multi-pass step: after the first pass, re-examine related files for cascading effects.

Q3: All findings real but too vague to act on (e.g. "possible issue")

Agent is hedging; findings are technically correct but lack specificity. You can't reproduce or act on them.
Answer: Prompt layer. Tighten the output contract: "don't say 'possible' or 'might'; if unsure, don't report it." Add a vague-vs-specific example.

6 THE SCOREBOARD: Three Stages, Four Metrics

Stage	F1	$/run	steps	latency
0 · naive	0.00	0.00022	1.0	6s
1 · prompt + context	0.75	0.00015	1.0	5s
2 · loop + harness	0.75	0.00036	2.8	16s

- **Prompt + context**: F1 jumps from 0.00 to 0.75, for almost no extra cost. A real output contract plus the out-of-diff caller now in view.
- **Loop + harness**: F1 holds at 0.75 on this slice — Flash-lite already found these bugs by reasoning statically. What moves instead: steps 1.0 → 2.8, latency 5s → 16s. The agent now **verifies** each finding (runs the tests, re-grounds the cited line) before it ships — confidence has a price, and now it's bounded and predictable.
- Not every stage has to move F1. A stage can pay off in cost, in bounded latency, or in an audit trail instead.

6 The Decision Guide: Symptom → Layer

Symptom → Layer to fix

- Vague / malformed findings → **Prompt**
- Agent invents specifics (hallucination) → **Prompt**
- Misses things that need more info → **Context**
- Slow / expensive → **Context**
- Can't do multi-lookup tasks → **Loop**
- Loops forever → **Loop**
- Unsafe actions taken → **Harness**
- Confidently wrong findings → **Harness**
- Unpredictable cost → **Harness**
- Needs human review on risky calls → **Harness**

Core principle

They compose. They don't substitute.
A better prompt doesn't fix a cost blow-up. A cost ceiling doesn't improve recall. Each layer solves a specific problem.

6 Adjacent Concepts

These are related but separate

- **Evaluation**: build the eval first. Score your agent on a fixed benchmark.
- **Determinism**: temperature, random seeds. Reproducibility for debugging.
- **Prompt injection**: Week 7. Untrusted input in context can override instructions.
- **Multi-agent orchestration**: Week 8. Delegating to specialist sub-agents.
- **Model choice per step**: cheap model to route, strong model to synthesise.
- **Guardrails**: input/output filters. Content policy enforcement.
- **Where frameworks fit**: LangGraph, LlamaIndex, AutoGen give loop + harness primitives. They don't do the prompt and context work for you.

Thank You!

Four layers. One agent, improved three times. A different number moved each time. Go build something that survives production.



Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design